AIMLCZG546 · SESSION 3 · COMPANION READER

Requirements Engineering for ML Systems

When to Use ML, Goals, GR4ML Notation, and Measuring Success

Session 3 · Read alongside the lecture slides · Every concept worked out in full

How to use this companion. The slides give you the formulas. This reader gives you the *why*. Every concept is expanded in plain English. Every example is worked out in full. Colour-coded boxes guide you: **blue** = core idea. **Amber** = real-life analogy. **Green** = worked scenario. **Red** = common mistake. **Purple** = the one sentence to remember. The running example is a *credit risk prediction system* at a bank.

1 When to Use ML

ML is a powerful tool. But it is not the right tool for every problem. Geoff Hulten's framework names three situations where ML wins over rule-based systems.

☞ The intuition

A screwdriver is perfect for screws but useless for nails. ML fits certain types of problems exactly. Using it on the wrong problem wastes months of effort. It can produce worse results than a simple if-else rule.

★ Everyday picture

Imagine a factory that sorts products by colour. A simple colour sensor beats any ML model. But if the factory needs to spot a defect that shifts form every month, ML adapts; the fixed sensor does not. The key question is: does the problem have stable, writable rules? If yes, write the rules. If no, use ML.

1.1 Case 1: Intrinsically Hard Problems

Human language is the classic example. Consider sarcasm: "Oh great, another Monday." A rule-based system sees the word "great" and flags it as positive. A human immediately reads it as negative. Language is contextual, ambiguous, and culturally loaded. No finite set of hand-crafted rules can capture all of that. ML models learn the patterns from millions of examples.

1.2 Case 2: Big Problems (Scale)

Recommending a song from 100 million tracks to 500 million users is too large for any human-written rule set. The combinations are astronomical. ML learns a compressed representation for each user and each song. It then finds the best match in milliseconds.

1.3 Case 3: Time-Changing Problems

Fraud patterns shift constantly. As soon as a rule blocks one attack, fraudsters adapt. A rule like "flag transactions above $50,000 at night" becomes outdated within weeks. An ML model retrained weekly adapts to new attack patterns. A fixed rule set does not.

✎ Worked example: Choosing between rule-based and ML

Problem A: Block purchase if the customer is under 18.

Step 1. Write a rule. `if age < 18: block`. Done.

Step 2. Is ML needed? No. The rule is clear, stable, and deterministic.

Step 3. Verdict: Rule-based wins. ML adds complexity with no benefit here.

Problem B: Detect new types of bank fraud in real time.

Step 1. Try a rule. Fraud evolves weekly. Any rule is outdated fast.

Step 2. Is ML needed? Yes. Fraud is a time-changing problem (Case 3).

Step 3. Verdict: ML wins. Retrain weekly on the latest fraud cases.

✗ Watch out

Using ML on a simple, deterministic problem is a mistake. If you can write a clear business rule, write it. ML adds cost, complexity, and uncertainty you do not need. Always ask: can a ten-line rule do this job?

Where this shows up in ML. Every ML project starts with a justification: *why ML and not a rule?* The answer must fall into one of Hulten's three cases. If it does not, the project should be reconsidered before any code is written.

✓ Key takeaway

Use ML for problems that are too complex for rules, too large to enumerate by hand, or too fast-changing to maintain manually. For everything else, a simple rule is cheaper and more reliable.

2 Requirements Engineering for ML Systems

Requirements Engineering (RE) is the process of discovering and documenting what a system must do. It also manages how well the system must do it. Traditional RE is straightforward: systems are deterministic. An ML system is probabilistic, data-dependent, and continuously changing. This makes RE much harder.

☞ The intuition

Traditional RE is like writing a recipe. ML RE is like describing the taste of a dish that changes every time you cook it. The freshness of the ingredients affects the result each time. The goal is still a good dish — but the specification must account for variability.

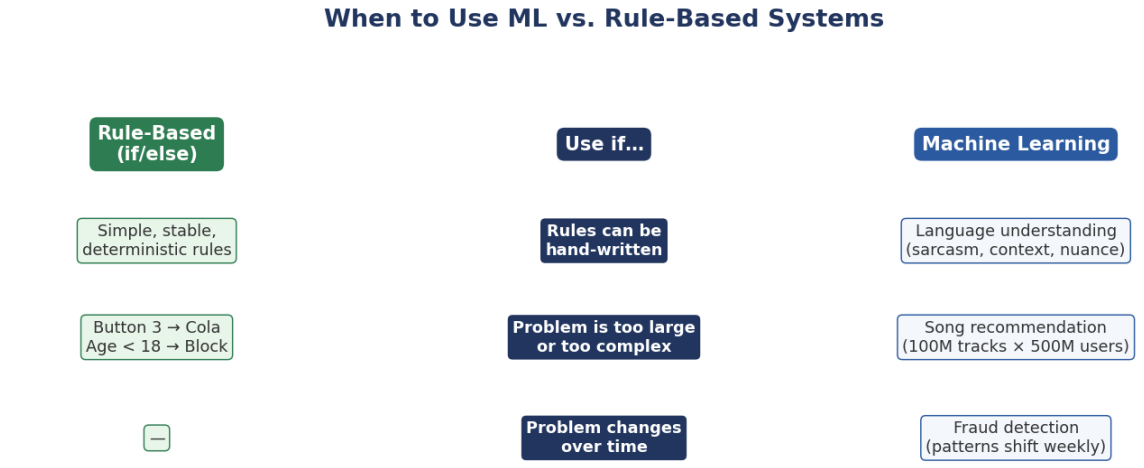

Figure 1: When to use ML versus rules. The three cases (hard, big, changing) are where ML wins. Anything else — small, stable, deterministic — is better served by a rule.

★ Everyday picture

A traditional vending machine always returns the correct product. The rule is: button 3 delivers Cola. An ML system is more like a barista who recommends drinks based on your mood. They learn from experience. But their behaviour varies. Requirements must capture both the goal and the expected variation.

Dimension	Traditional RE	ML RE
Specification	Clear, exhaustive rules	Measurable targets on data
Correctness	Binary (pass/fail)	Probabilistic (93 % correct)
Data dependency	None	All outputs depend on training data
Change	When business rules change	When data distribution shifts

RE in ML must address four concerns not found in traditional RE. First, data quality and availability. Second, model behaviour under distributional shift. Third, continuous learning and retraining cycles. Fourth, fairness and bias across user groups.

Where this shows up in ML. In practice, ML requirements documents look different from traditional ones. They include data schemas, data quality thresholds, model performance contracts, and retraining schedules. They are living documents that change as data changes.

✓ Key takeaway

ML RE must specify *what* the system does, *on which data*, and *how often* it retrains. Both the world and the model's behaviour drift over time.

3 Goals in ML Systems

A **goal** is a desired outcome. In an ML system, goals must line up across four levels at the same time.

☞ The intuition

Think of a telescope. Each lens must be aligned for the final image to be sharp. If the model is accurate but the system is slow, users are unhappy. If users are happy but the system costs too much, the business is unhappy. All four levels must point in the same direction.

★ Everyday picture

A football team has four levels of goal. The club wants to win the league (organisational). Fans want exciting matches (user). The coaching staff want consistent performance (system). Players want a clear role and high pass-completion (model). If any level is ignored, the team underperforms even if the others are fine.

The four levels are:

- **Organisational goals** — cost, revenue, compliance, growth.
- **User goals** — satisfaction, trust, speed, ease of use.
- **System goals** — latency, availability, throughput, reliability.
- **Model goals** — accuracy, F1 score, AUC, false-negative rate.

Goals at lower levels are *derived from* the levels above. You cannot sensibly set a model goal without first knowing the business goal.

✎ Worked example: Goal alignment in the credit risk system

A bank wants to approve loan applications faster and reduce default rates. We derive each goal level from the one above.

Step 1. **Organisational goal.** Reduce loan default rate by 15 % over 12 months. Cut manual review cost by 40 %.

Step 2. **User goal (loan officer).** Get a risk score with an explanation within 10 seconds of submission.

Step 3. **System goal.** Process the risk scoring API in under 2 seconds. Maintain 99.9 % availability.

Step 4. **Model goal.** Reach AUC ≥ 0.85 on the held-out test set. Keep the false-negative rate $\leq 5\,\%$, so true defaults are not missed.

Step 5. **Sense-check.** The model goal (AUC, FNR) flows directly from the organisational goal (reduce defaults). If you set the model goal without the business goal, you might optimise accuracy and still fail in production.

✗ Watch out

Optimising only the model goal (AUC) while ignoring system goals is a classic mistake. A model with AUC 0.92 that takes 30 seconds to respond is useless. The loan officer is waiting at the counter with a customer.

Goals across levels are not always aligned. They may also conflict.

- **Support each other:** Higher accuracy often raises user satisfaction.
- **Conflict:** High accuracy needs a large model; a large model is slow (accuracy vs. latency). Better quality costs more (quality vs. cost).

When goals conflict, the team must make explicit trade-offs. Write the trade-off decision down. It is a requirements decision, not a code decision.

Where this shows up in ML. This goal hierarchy directly maps to ML system design. Organisational goals drive the loss function choice. User goals drive latency budgets and UI decisions. System goals drive infrastructure choices (batch vs. real-time). Model goals drive architecture and training choices.

✓ **Key takeaway**

Always derive model goals from business goals, not the other way around. A model with a great AUC that fails a business goal is a failed project.

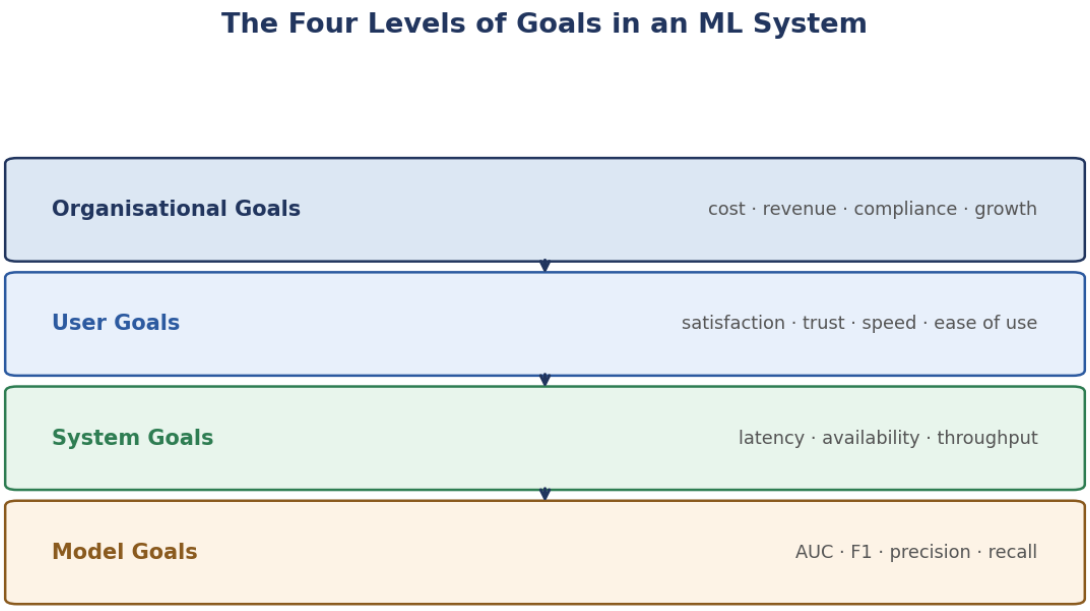

Figure 2: The four levels of goals in an ML system. Organisational goals sit at the top and drive all layers below. Model goals (AUC, F1) are the bottom layer — derived from the layers above.

4 From Goals to Requirements: The GR4ML Connection

Once you know the goals, how do you move to concrete ML requirements? The path has three steps.

Step 1. **Identify the decisions.** Look at what decisions people make to reach their goals. In the bank: "Should I approve this loan?"

Step 2. **Find which decisions need predictions.** The approval decision needs a credit risk score. That is a *prediction* task.

Step 3. **Check if ML can support that prediction.** Is there labelled historical data (past

loans with outcomes)? Is the pattern too complex or too large for hand-written rules? If yes, ML is appropriate.

☞ The intuition

Goals tell you *what* you want. Decisions tell you *what choices* get you there. Predictions tell you *what information* those choices need. ML is the engine that produces the predictions.

This chain (Goal → Decision → Question → Prediction → ML) has been formalised as the **GR4ML** notation, which we explore in the next three sections.

✓ Key takeaway

Requirements engineering for ML starts with goals. Then it finds the decisions those goals require. Then it asks which predictions those decisions need. Only then does it design an ML model.

5 GR4ML: What It Is and Why We Need It

GR4ML (Goals and Requirements for Machine Learning) is a conceptual modelling framework. It connects high-level business goals to the specific ML components that serve them.

☞ The intuition

GR4ML is like an architect's blueprint. The blueprint connects the client's wish ("I want a large kitchen") to the builder's plan (beams, pipes, wiring, wall positions). Without the blueprint, the builder guesses. Without GR4ML, the ML engineer guesses.

Without GR4ML, organisations often build ML systems that:

- Optimise the wrong metric (AUC when the goal is user retention).
- Use the wrong type of analytics (predicting when the need is prescribing).
- Build on poorly prepared data (garbage in, garbage out).

GR4ML avoids these problems by forcing alignment before any code is written. It has three views, each answering a different question:

View	Question	Covers
Business View	Why do we need ML?	Actors, goals, decisions, indicators
Analytics Design View	What analytics is needed?	Goals, algorithms, quality trade-offs
Data Preparation View	How to prepare the data?	Sources, cleaning, feature engineering

Where this shows up in ML. GR4ML is a requirements tool, not a code tool. It runs before any model is built. Teams that skip it often spend weeks rebuilding models because the first version optimised the wrong thing.

✓ Key takeaway

GR4ML aligns three layers: business goals, analytics choices, and data preparation. All three must align before any model is trained.

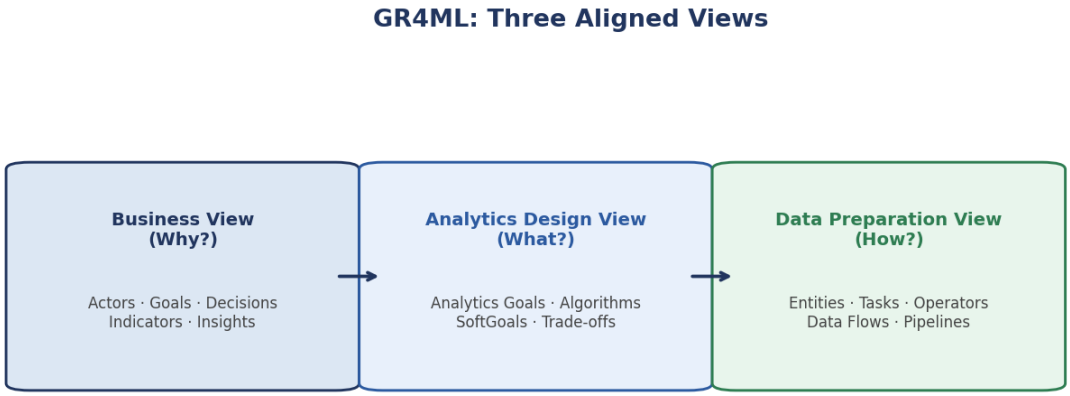

Figure 3: The three GR4ML views. Business View answers "why". Analytics Design View answers "what". Data Preparation View answers "how". Each view feeds into the next.

6 GR4ML: Business View

The **Business View** maps the organisational context of the ML system. It answers the question: *who needs ML, and why?*

☞ The intuition

The Business View is a "why" map. It traces the chain from "why do we need this system at all?" (StrategicGoal) down to "what specific number does the model produce?" (Insight). Every ML decision in this chain can be justified by a business need.

★ Everyday picture

A hospital wants to reduce patient readmissions. The strategic goal is "reduce 30-day readmission rate by 20 %". The decision goal is "decide whether to arrange a follow-up call for this patient". The question goal is "what is the readmission risk for this patient?" The insight is "readmission risk score: 0.78 (high risk)". The ML model produces that one number.

The main elements of the Business View are:

Element	Meaning	Credit risk example
Actor	Stakeholder who drives decisions	Case Worker (loan officer)
StrategicGoal	High-level business objective	Make good lending decisions
DecisionGoal	Specific decision to fulfil the strategy	Approve or reject this loan
QuestionGoal	Question that must be answered to decide	What is the credit risk?
Insight	Data-driven answer to the question	Credit risk score: 0.73 (high risk)
Indicator	Metric measuring if the goal is met	Loan default rate (%)
UpdateFrequency	How often the model runs	Monthly (batch retraining)
LearningPeriod	Historical window for training	Last 24 months of loan data

✎ **Worked example: Business View for credit risk prediction**

The bank wants to lend money safely and quickly. We model the Business View step by step.

Step 1. **Identify the Actor.** The Case Worker reviews each loan application. They make the approve/reject decision.

Step 2. **State the StrategicGoal.** "Make good lending decisions quickly and safely." This is high-level and not yet measurable.

Step 3. **Break into DecisionGoal.** "Decide: approve or reject this specific loan application." This is a concrete, recurring decision.

Step 4. **Ask the QuestionGoal.** "What is the credit risk of this applicant?" This is the question the ML model must answer.

Step 5. **Define the Insight.** The model outputs a credit risk score (0.0 = safe, 1.0 = high risk). Score above 0.7 triggers a manual review flag.

Step 6. **Set the Indicator.** Loan default rate (%) — measured monthly — tracks whether the StrategicGoal is being achieved in production.

Notice that the QuestionGoal directly defines the model's output. The StrategicGoal and Indicator define how we evaluate success in the real world.

✗ **Watch out**

Mixing up the four element types is common. A QuestionGoal is not the same as an Indicator. The QuestionGoal is what the model answers for each input ("is this loan risky?"). The Indicator is the aggregate metric measured over time ("default rate is 4.2 %"). Confusing them leads to models that optimise the wrong target.

Where this shows up in ML. In practice, the Business View maps directly to the ML pipeline. The QuestionGoal defines the prediction target. The Insight defines the output format. The Indicator defines the production monitoring metric. Teams that skip this step often retrain models because the initial target was the wrong one.

✓ **Key takeaway**

The Business View chains: StrategicGoal → DecisionGoal → QuestionGoal → Insight. The ML model is built to answer the QuestionGoal.

7 GR4ML: Analytics Design View

The **Analytics Design View** answers the question: *what type of analytics does the ML component need to perform?* It specifies the analytics goal, picks the algorithm, and records the quality trade-offs.

☞ **The intuition**

If the Business View says "we need a credit risk score", the Analytics View decides *how* to produce it. Is this a prediction? A description? A prescription? Choosing the wrong type is like using a hammer when you need a saw. Both are tools; neither can do the other's job.

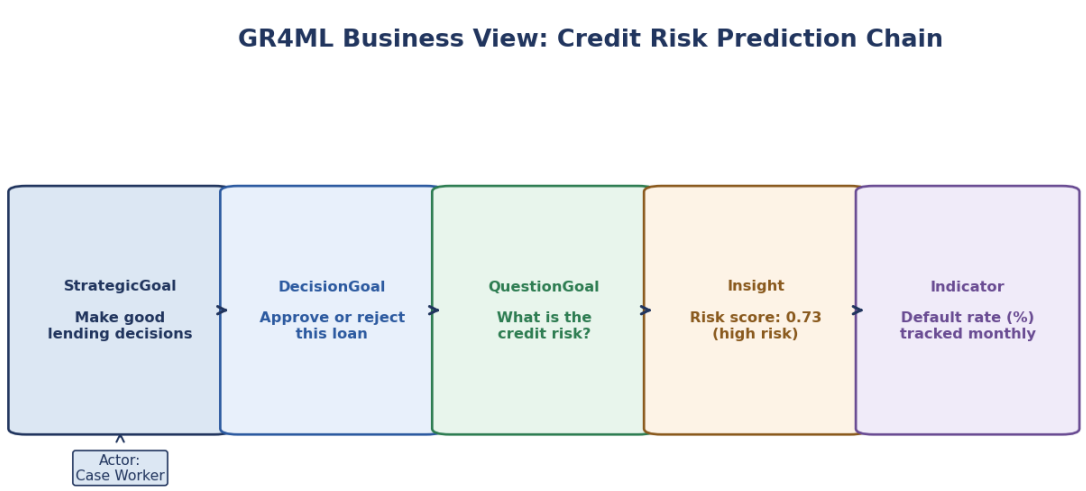

Figure 4: The Business View chain for credit risk prediction. Each box is one GR4ML element. The arrows show how the StrategicGoal drives all elements below it. The ML model sits at the Insight level: it answers the QuestionGoal.

The three types of analytics goals are:

- **PredictionGoal** — forecast a future or unknown value. Example: "Will this applicant default?"
- **DescriptionGoal** — understand or summarise existing data. Example: "Which customer segments default most?"
- **PrescriptionGoal** — recommend the best action. Example: "Which loan terms minimise expected loss?"

A **SoftGoal** is a quality attribute that guides algorithm choice. It is not a hard requirement. It is a preference. Examples: interpretability, fairness, speed, cost of retraining.

✎ Worked example: Analytics design trade-off for credit risk

The bank's model must be both *accurate* and *interpretable*. Accurate means: minimise defaults (AUC target ≥ 0.85). Interpretable means: loan officers must explain rejections to customers — this is a regulatory requirement. We evaluate two algorithms.

Step 1. **Option A: XGBoost.** AUC = 0.88. Very accurate. But it is a black-box model. The loan officer cannot explain to the customer why they were rejected. SoftGoal: interpretability = LOW. Not acceptable.

Step 2. **Option B: Logistic Regression.** AUC = 0.81. Less accurate. Fully interpretable: each feature has a named weight. SoftGoal: interpretability = HIGH. Meets the regulatory requirement. But AUC 0.81 is below the 0.85 target.

Step 3. **Option C: XGBoost + SHAP.** AUC = 0.88. High accuracy. SHAP (SHapley Additive exPlanations) produces a local explanation for each prediction: "Rejected because debt-to-income ratio 45 % exceeds threshold 35 %." SoftGoal: interpretability = HIGH via post-hoc explanation.

Step 4. **Decision.** Choose Option C. It meets both the accuracy SoftGoal (≥ 0.85) and the interpretability SoftGoal (post-hoc SHAP explanation).

✗ Watch out

SoftGoals are not optional. They capture real constraints (regulatory, ethical, operational). An algorithm with high accuracy but low interpretability can fail a compliance audit even if it predicts defaults correctly. Always document SoftGoals before choosing an algorithm.

Where this shows up in ML. The Analytics View directly drives the model selection process. The AnalyticsGoal type determines the model family. Prediction goals need supervised classifiers. Description goals need clustering or summarisation. Prescription goals need reinforcement learning or optimisation. SoftGoals constrain the search space (interpretable, fast, cheap, fair).

✓ Key takeaway

The Analytics View decides *what* the model must do. It also records how good the model must be on each quality dimension: accuracy, speed, interpretability, and fairness. Document all of this before picking an algorithm.

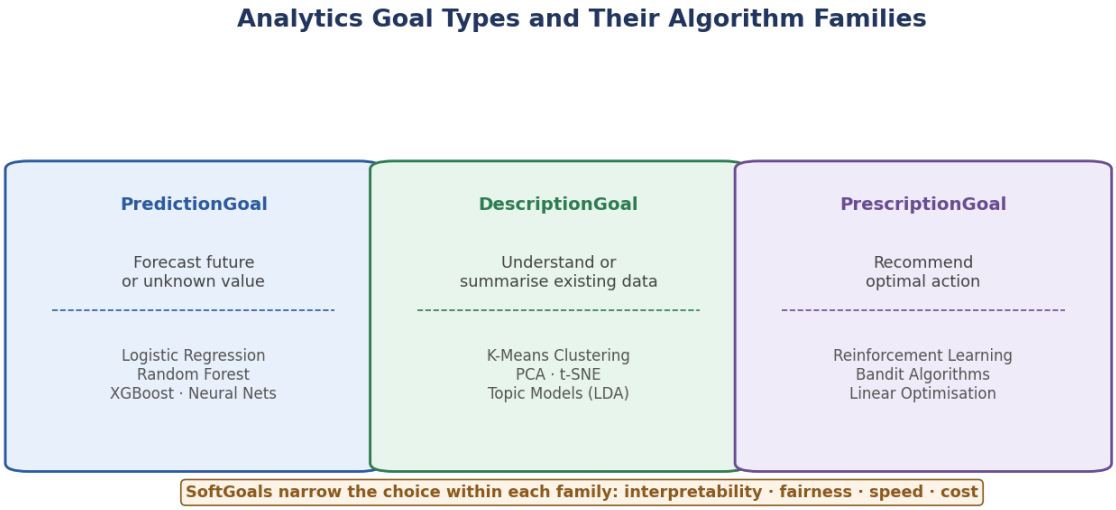

Figure 5: Three analytics goal types and their algorithm families. A PredictionGoal needs a supervised classifier or regressor. A DescriptionGoal needs clustering or summarisation. A PrescriptionGoal needs reinforcement learning or optimisation. SoftGoals narrow the choice within each family.

8 GR4ML: Data Preparation View

The **Data Preparation View** answers the question: *how do we prepare the data so the model can learn from it?* It maps raw data sources to the clean, feature-engineered dataset the model trains on.

☞ The intuition

Data preparation is like preparing ingredients before cooking. You cannot make a good meal from rotten vegetables or badly measured quantities. The Data Preparation View is the shopping list and the prep instructions. It names every ingredient, every step, and every tool.

★ Everyday picture

A chef gets fresh fish, potatoes, and salt. They must fillet the fish (cleaning), dice the potatoes (reduction), add salt to taste (normalisation), and plate it nicely (encoding for presentation). Each operation transforms the raw ingredient into something the dish can use. Data preparation is exactly that chain of transformations.

The main elements of the Data Preparation View are:

- **Entity** — a data object or table: `LoanApplication`, `CreditBureau`.
- **DataPreparationTask** — a transformation: cleaning, reduction, normalisation, encoding.
- **Operator** — the specific method used: remove rows with missing income, min-max scale loan amount, one-hot encode loan purpose.
- **DataFlow** — the sequence: raw data → task → cleaned data → next task.

✎ Worked example: Data preparation for the credit risk model

Input: Three raw tables — LoanApplication (50 columns), CreditBureau (30 columns), and three years of TransactionHistory.

Step 1. **Cleaning.** Remove 3,200 rows with missing annual income (3.2 % of 100,000 records). Impute 800 rows with missing credit score using the median by age bracket. Result: 96,800 clean rows.

Step 2. **Reduction.** Drop 22 columns with more than 40 % missing values. Those columns carry too little signal to be useful. Result: 58 columns remain.

Step 3. **Normalisation.** Apply min-max scaling to: loan_amount, annual_income, total_debt. This brings all three to the range $[0, 1]$. Formula: $x' = (x - x_{\min})/(x_{\max} - x_{\min})$.

Step 4. **Encoding.** One-hot encode loan_purpose (12 categories → 12 binary columns). One-hot encode employment_type (4 categories → 4 binary columns). Result: 70 features total.

Step 5. **Output.** 97,000 rows, 70 features, ready for XGBoost training. The Indicator (loan default rate) is the label column.

✗ Watch out

Data preparation errors are the most common cause of model failure in production. Normalisation parameters (min, max) must be computed on the *training set only*, then applied to validation and test sets. If you normalise on all data first, you leak future information into training. This inflates your measured performance. It will not hold in production.

Where this shows up in ML. The DataPreparationTask list is the basis of the ML pipeline code. Each task maps to a step in a Scikit-learn Pipeline or a Spark job. The Operator specifies which function to call (e.g. `StandardScaler`, `OneHotEncoder`). Documenting this view before coding prevents data leakage and makes pipelines reproducible.

✓ Key takeaway

The Data Preparation View is not just documentation. It is the blueprint for the ML pipeline code. Every operator must be reproducible and must never use future data.

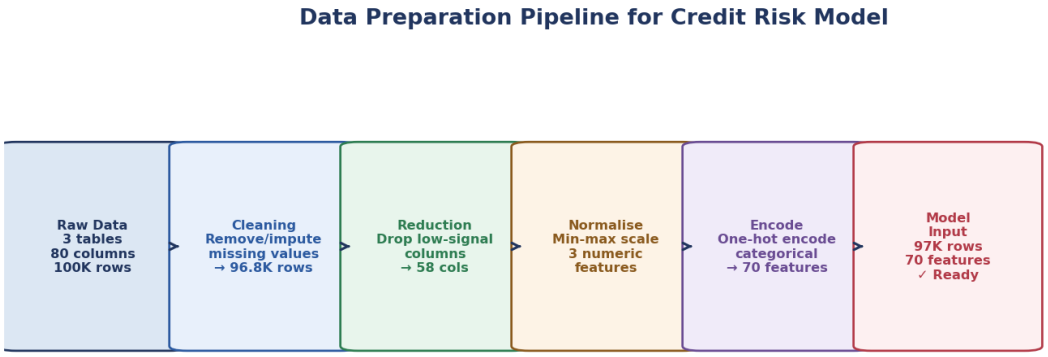

Figure 6: A data preparation pipeline for the credit risk model. Raw data enters from the left. Each task transforms it. The output is a clean, encoded, normalised feature matrix ready for training.

9 Measuring Goals: Measures and Metrics

A **measure** is the method used to quantify something. A **metric** is the actual measured value. A good measure: directly relates to a goal, can be counted or scored, and is practical to collect in production.

☞ The intuition

A goal without a measure is just a wish. "Improve chatbot usefulness" is a wish until you say: "Task success rate above 80 %." The measure turns the wish into a target you can test.

★ Everyday picture

A measure for "good health" might be blood pressure, temperature, and resting heart rate. The metric for your blood pressure today is 120/80 mmHg. The goal (good health) is abstract. The measure (blood pressure) makes it concrete. The metric (120/80) tells you where you stand today.

A measure should:

- **Relate directly to a goal.** Not a proxy of a proxy.
- **Be quantifiable and objective.** Anyone who collects it gets the same number.
- **Be practical to collect.** Theoretical measures that are impossible to gather in production are useless.

✎ **Worked example: Chatbot goal to measures**

Goal: "Improve chatbot usefulness."

Step 1. **Measure 1 — task success rate.** Fraction of conversations where the user's intent was fulfilled. Collected by: user logs + intent detection. Target: $\geq 80\,\%$ of conversations.

Step 2. **Measure 2 — satisfaction score.** User rates the chat 1–5 stars at the end. Collected by: post-chat survey. Target: average ≥ 4.0 stars.

Step 3. **Measure 3 — conversation completion rate.** Fraction of users who did not abandon mid-conversation. Collected by: session logs. Target: $\geq 70\,\%$ of sessions.

Step 4. **Measure 4 — escalation rate.** Fraction of conversations escalated to a human agent. Collected by: escalation logs. Target: $\leq 15\,\%$ (lower is better).

Step 5. **Sense-check.** "Improve chatbot usefulness" is still unmeasurable by itself. Only after defining these four measures can you set targets, run experiments, and know if you are improving.

✗ **Watch out**

Do not confuse model metrics (AUC, F1) with business measures (task success rate, default rate). Model metrics tell you how the model performs on a test set. Business measures tell you whether the product goal is met in production. A model with AUC 0.9 can still fail the business measure. This happens when the threshold is wrong, or when production data differs from training data.

Where this shows up in ML. In ML systems, goals map to a hierarchy of measures. Organisational goals map to business KPIs (default rate, conversion rate). User goals map to UX metrics (satisfaction, completion, escalation). Model goals map to ML metrics (AUC, F1, RMSE). All three must be tracked in production, not just the ML metrics.

✓ **Key takeaway**

Define measures before building the model. If you cannot measure it, you cannot improve it. Business measures and model metrics must both be tracked in production.

10 Accuracy vs Precision of Measurement

These two words are often confused. They describe separate properties of a measurement system. They are *independent*: a system can be accurate but not precise, precise but not accurate, both, or neither.

- **Accuracy** — how close the measured value is to the true value. Low accuracy means systematic *bias*: the measurement is always off in one direction.
- **Precision** — how consistently the measurement gives the same result when repeated. Low precision means high *variance*: results scatter widely even when the true value is unchanged.

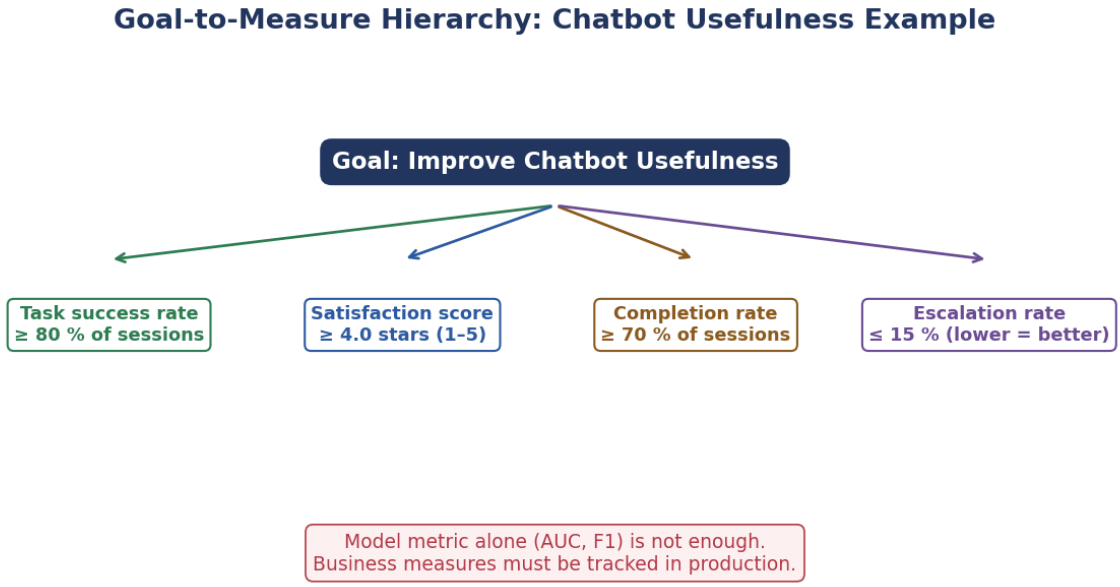

Figure 7: Goal-to-measure hierarchy for the chatbot example. The abstract goal at the top breaks down into four concrete measures. Each measure has a target and a collection method. A model metric alone is not enough; business measures must also be tracked.

☞ The intuition

Think of darts. **Accurate** = clustered around the bullseye (right place, right answer). **Precise** = clustered tightly together (consistent, but may miss the bullseye). The ideal is both: tight cluster centred on the bullseye. The worst case is neither: darts scattered all over the board.

★ Everyday picture

A bathroom scale that always reads 2 kg too heavy is precise (consistent) but not accurate (wrong). A scale that reads differently every time you step on it is neither accurate nor precise. You want a scale that reads correctly and reads the same each time.

✎ Worked example: Thermometer accuracy and precision

True body temperature: 98.6°F. We take five readings with two thermometers.

Step 1. **Thermometer A readings:** 98.2, 98.1, 98.3, 98.2, 98.1. Mean = 98.18°F. Bias = 98.18 − 98.6 = −0.42°F. Standard deviation = 0.08°F. **Verdict:** Very consistent (low SD) but consistently wrong (high bias). This is *precise but not accurate*. Calibration (adding 0.42 to every reading) can fix the bias.

Step 2. **Thermometer B readings:** 98.4, 99.1, 97.9, 98.8, 99.2. Mean = 98.68°F. Bias = 98.68 − 98.6 = +0.08°F. Standard deviation = 0.53°F. **Verdict:** Centred on the right value (low bias) but noisy (high SD). Hard to rely on for a single reading. This is *accurate but not precise*.

Step 3. **ML connection.** In ML, a model with high accuracy (low bias) makes correct

predictions on average. A model with high precision (low variance) makes consistent predictions. The bias–variance trade-off is the exact ML counterpart of accuracy vs. precision.

✗ Watch out

In ML, the word "accuracy" usually means the fraction of correct classifications. That is a model metric, not the measurement-theory concept of accuracy here. Do not mix the two. Measurement precision (consistency) and model accuracy (fraction correct) are different quantities.

Where this shows up in ML. When you measure a goal metric in production (e.g. user satisfaction), you need *measurement accuracy*: does the survey capture true satisfaction? You also need *measurement precision*: do repeated surveys give similar results? If your measurement instrument is biased, your model optimises the wrong signal. This is a common and hard-to-detect failure mode.

✓ Key takeaway

Precision without accuracy means measuring the wrong thing reliably. Accuracy without precision means you cannot trust any single reading. You need both. In ML: high-precision / low-variance and high-accuracy / low-bias.

Glossary & Symbols

Key terms.

Requirements Engineering
: The process of finding, documenting, and managing what a system must do and how well it must do it.

Goal
: A desired outcome of a system. In ML, goals span four levels: organisational, user, system, and model.

GR4ML
: A conceptual modelling framework that connects business goals to ML components through three views: Business, Analytics Design, and Data Preparation.

StrategicGoal
: The high-level business objective an organisation wants to reach.

DecisionGoal
: A specific decision that must be made to fulfil the strategic goal.

QuestionGoal
: The question that must be answered to support the decision.

Insight
: The data-driven answer the ML model produces (e.g. a risk score).

Indicator
: A metric that tracks whether the strategic goal is met over time.

AnalyticsGoal
: The type of analysis required: prediction, description, or prescription.

SoftGoal
: A quality preference (interpretability, fairness, speed) that guides algorithm choice without being a hard constraint.

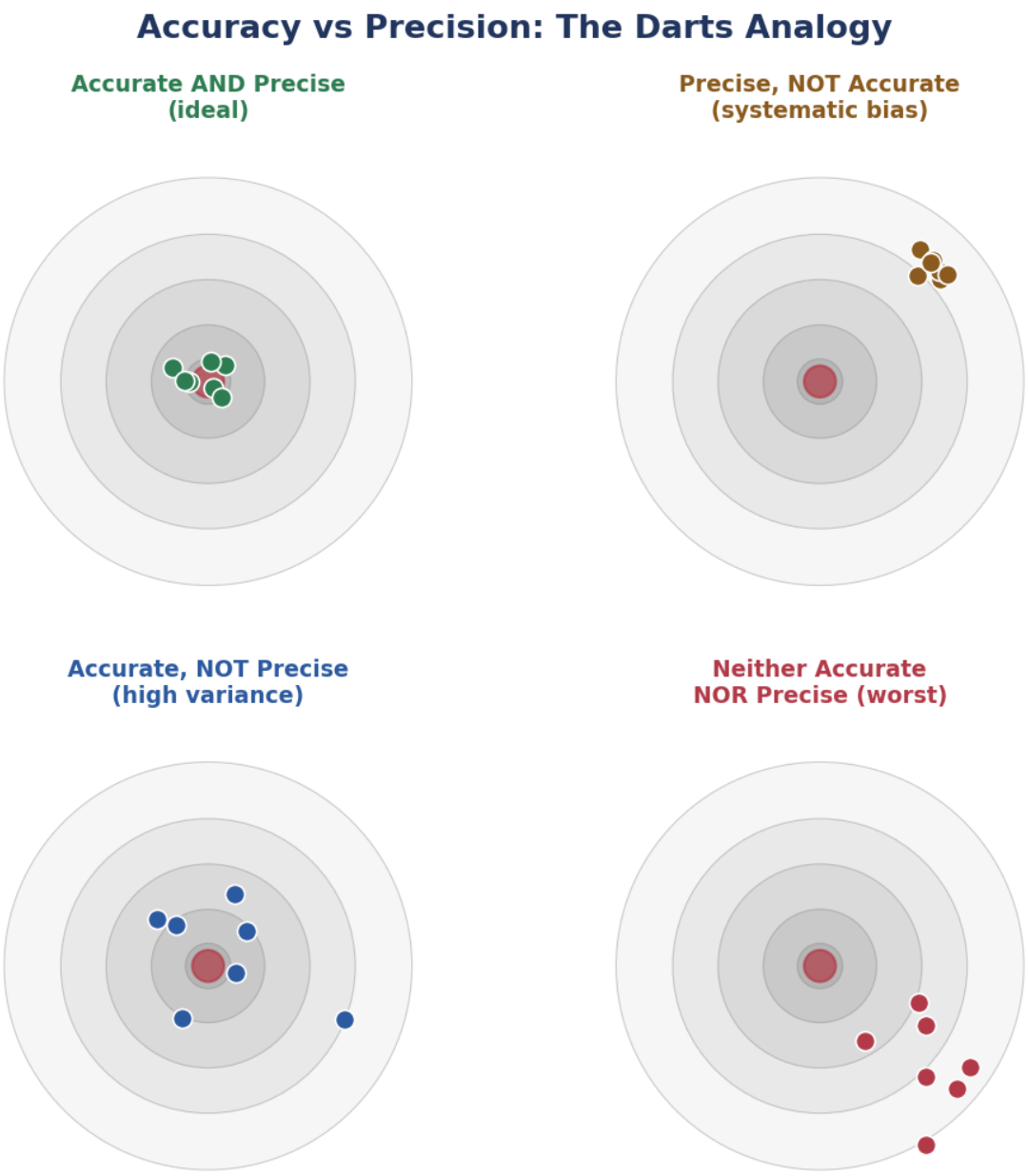

Figure 8: The four combinations of accuracy and precision. Top-left: accurate and precise (ideal). Top-right: precise but not accurate (systematic bias). Bottom-left: accurate but not precise (high variance). Bottom-right: neither accurate nor precise (worst case).

DataPreparationTask
A transformation applied to raw data: cleaning, reduction, normalisation, or encoding.

Measure A method or standard used to quantify a goal.

Metric The actual measured value at a given time.

Accuracy (measurement)
How close a measured value is to the true value. Low accuracy = systematic bias.

Precision (measurement)
How consistently a measurement gives the same result on repeat. Low precision = high variance.

Symbols at a glance.

Symbol	Meaning
AUC	Area under the ROC curve; measures binary classifier quality
F1	Harmonic mean of precision and recall; balances the two
x'	Normalised feature value: $(x - x_{\min})/(x_{\max} - x_{\min})$
$x_{\min}$, $x_{\max}$	Minimum and maximum of the feature in the training set
$\bar{x}$	Mean (average) of a set of measurements
SD (std dev)	Standard deviation; measures spread of repeated measurements